AI Academic Management System

Project Specification, Roadmap & Feature Plan

A full-stack academic management platform (Spring Boot + MySQL + FastAPI) with JWT authentication, role-based access, and AI-assisted features. Designed to be built in phases, starting from a working MVP and layering AI/ML features on top.

Target use	PFA / Engineering School Project (2nd Year)
Also used for	GitHub portfolio project, LinkedIn showcase, Resume/CV entry
Core stack	HTML/CSS/JS → Spring Boot (API) → MySQL, + FastAPI (AI/ML)
Approach	Build MVP first (fully working end-to-end), then add AI features one by one

1. Architecture Overview

Spring Boot is the brain of the system: it owns authentication, business logic, and all core CRUD operations. FastAPI is a separate, focused service that only handles AI and machine learning tasks. The frontend talks only to Spring Boot; Spring Boot talks to FastAPI when AI/ML results are needed.

Layer	Technology	Responsibility
Frontend	HTML + CSS + JavaScript (fetch API)	UI, calls REST endpoints, stores JWT
Backend API	Spring Boot	Auth, CRUD, business logic, security, orchestration
Database	MySQL	Persistent storage (users, grades, attendance, etc.)
AI/ML Service	FastAPI	Course generation, quiz generation, chat assistant, pass prediction

Request flow:

```
Browser (fetch, JSON) → Spring Boot REST API → MySQL (via JPA/Hibernate)
Spring Boot → FastAPI (HTTP call) → ML model / LLM prompt → JSON result back to Spring Boot
→ Frontend
```

2. User Roles & Features

Director (Admin)

- Login (JWT)
- Create / edit / delete teachers
- Create / edit / delete students
- Create classes and modules
- Assign teachers to modules
- View overall statistics dashboard (averages, attendance, pass rate)
- Export reports (PDF/CSV)
- Receive alerts (low attendance, low average, at-risk students)

Teacher

- Login (JWT)
- View assigned classes and modules
- Add / edit grades for students
- Add / edit attendance records
- Upload course files (PDF, slides)
- Generate a course outline using AI
- Generate a quiz (MCQ / True-False / coding question) using AI

Student

- Login (JWT)
- View personal timetable
- View personal grades
- View personal attendance
- Download course documents
- Ask the AI assistant course-related questions

3. Database Schema (MySQL)

Roughly 10–12 tables — enough to practice real relational design (one-to-many, many-to-one, and one many-to-many via TeacherModule).

users

id (PK) • username • password (hashed) • role (DIRECTOR / TEACHER / STUDENT)

student

id (PK) • firstname • lastname • birthdate • email • class_id (FK) • user_id (FK)

teacher

id (PK) • firstname • lastname • email • user_id (FK)

class

id (PK) • name • year

module

id (PK) • name • teacher_id (FK) • class_id (FK)

grade

id (PK) • student_id (FK) • module_id (FK) • grade • semester

attendance

id (PK) • student_id (FK) • date • present (boolean)

course_document

id (PK) • module_id (FK) • file_path • uploaded_at

ai_course_outline

id (PK) • module_id (FK) • content (text/JSON) • generated_at

ai_quiz

id (PK) • module_id (FK) • content (JSON) • generated_at

prediction

id (PK) • student_id (FK) • pass_probability • computed_at

Key relationships to practice:

Teacher → Module (@OneToMany) | Module → Class (@ManyToOne) | Class → Student (@OneToMany) | Student/Module → Grade (@ManyToOne on both sides)

4. Authentication (JWT)

Spring Security handles login, password hashing (BCrypt), and JWT issuance/validation. Every protected endpoint requires a valid token; the role inside the token controls what data/actions are allowed (Director / Teacher / Student).

```
Login request → Spring verifies password (BCrypt) → Spring creates JWT (signed, with role claim) → Frontend stores token (memory or localStorage) → Every request sends header Authorization: Bearer <token> → Spring filter validates token → request proceeds or 401/403
```

5. AI Features (added after the MVP works)

1. Course Outline Generator

Teacher clicks 'Generate Course'. Prompt example: "Create a 12-week Java course for first-year engineering students, 3 hours/week." AI returns a week-by-week outline, stored in the database (ai_course_outline table).

2. Quiz Generator

Teacher clicks 'Generate Quiz'. AI creates a mix of MCQ, True/False, and a coding question for the selected module, stored as JSON (ai_quiz table).

3. Student AI Assistant

Student asks a course question (e.g. "Explain inheritance"). Request goes through Spring Boot to FastAPI, which calls the LLM and returns an answer.

4. PDF Summarizer

Teacher uploads a course PDF; AI produces a short summary of the content.

Machine Learning (your own model, not an LLM)

A separate scikit-learn model (trained and saved with joblib, served by FastAPI) predicts each student's probability of passing, based on features such as: math grade, subject grades, attendance rate, assignment completion, and previous semester average. Output: a probability (e.g. 89%) shown on the director dashboard, with students flagged as at-risk below a threshold.

Beginner-friendly shortcut for v1: start with a simple rule (e.g. if attendance < 50% or average < 10 → flag as at-risk). It looks identical in the dashboard demo and lets you finish the MVP faster. Swap in the real trained model afterward as a documented 'phase 2' improvement.

6. Director Dashboard & Notifications

Charts (Chart.js): number of students/teachers/classes, average grade, attendance rate, pass rate.

ML/rule-based insights: predicted passing vs failing, list of at-risk students.

Notification examples: attendance below 50% → alert teacher; student average below 10 → notify director.

Optional extras (only after MVP + one AI feature are done)

- QR-code based attendance
- Dark mode
- Search & sort on students/grades
- Export reports to PDF
- Email notifications to students
- Calendar view
- Announcements board
- Assignment submission
- Leaderboard / achievements

7. Folder Structure

```
School-System/
  frontend/
    html/
    css/
    js/
  spring/
    controller/
    service/
    repository/
    entity/
    dto/
    security/
    config/
  fastapi/
    app.py
    prediction.py
    model.joblib
  database/
  README.md
```

8. Development Roadmap (build in this order)

Phase	What to build	What you'll learn
Phase 1 — Authentication	Spring Security + JWT, login API, BCrypt password hashing.	Spring Security JWT
Phase 2 — Core CRUD	Student CRUD, Teacher CRUD, Module CRUD.	Spring Boot CRUD JPA
Phase 3 — Relationships	Wire up Teacher → Module → Class → Student with @OneToMany / @ManyToOne / @ManyToMany.	JPA relationships
Phase 4 — Frontend	Connect every page with fetch(): GET, POST, PUT, DELETE.	Fetch API + REST
Phase 5 — Prediction	Train a model (scikit-learn, joblib) or start rule-based; Spring → FastAPI → prediction → store result.	scikit-learn, FastAPI
Phase 6 — Statistics	Build the director dashboard with Chart.js: averages, pass rate, attendance, predictions.	Chart.js
Phase 7 — AI Features	Connect an LLM API for course generation, quiz generation, and the student assistant.	LLM API integration
Phase 8 — Deployment	Git/GitHub, then Docker later; deploy via Render or Railway.	Git, Docker, Render/Railway

9. MVP Scope — Build This First, End-to-End

Do not build every feature at once. Get this working completely before adding anything else:

- Login with JWT for all 3 roles
- Director manages students, teachers, modules, and classes
- Teachers enter grades and attendance
- Students log in and see only their own information
- Simple pass/fail risk flag (rule-based, or FastAPI + scikit-learn if time allows)
- Director dashboard with basic charts (average grades, attendance, pass rate)

Once the MVP works end-to-end, add AI features one at a time, in this order:

- 1. AI-generated course outlines
- 2. AI-generated quizzes
- 3. Student AI assistant
- 4. PDF summarization

Beginner-polish additions (cheap, high impact)

- Input validation + clear error messages on every form
- .env config + a proper README.md with setup steps and screenshots/GIF
- Pagination on student/teacher lists
- Swagger / OpenAPI docs for the Spring Boot API
- Correct CORS configuration between frontend and backend

10. Technologies You Will Learn

Category	Technologies
Frontend	HTML, CSS, JavaScript, Fetch API, Chart.js
Backend	Spring Boot, Spring Security, JWT, JPA, Hibernate, MySQL
AI / ML	FastAPI, scikit-learn, joblib, LLM API integration
General	REST APIs, authentication, database design, file uploads, error handling

11. Project Rating (out of 10)

Field	Score /10	Notes
Technical depth	9	Auth, relational DB, REST, ML, LLM integration in one project
Learning value	10	Teaches JWT, JPA, REST design, ML basics, service-to-service calls
Resume / CV impact	9	Strong bullet point for a PFA-level application
GitHub appeal	8	Depends on a clean repo + good README + working demo
LinkedIn appeal	8	Best with a short demo video and clear caption
Portfolio appeal	8	One polished full project beats several shallow ones
Originality	6	Common project type; the AI/ML layer differentiates it
Feasibility on timeline	5	Main risk — mitigate by following the MVP-first order above
Interview talking points	9	Every feature maps to a real, explainable engineering decision
Overall as a PFA project	8	Build in phases, document well, treat AI as phase 2

12. Publishing Checklist

- GitHub: one clean repo, proper folder structure, README with architecture diagram, setup steps, screenshots/GIF
- LinkedIn: post once the MVP works end-to-end — short video demo + 3-4 sentences on what you built and learned
- Portfolio: same MVP demo, plus a clearly labeled 'Roadmap / In Progress' section for the AI features